Notes on Chapter 15 - Regular Expressions

Norman Lim

2025-07-29

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

```
library(babynames)
```

Pattern basics

- Using `str_view()` for basic string pattern matching:

```
str_view(fruit, "berry")
```

```
##  [6] | bil<berry>
##  [7] | black<berry>
## [10] | blue<berry>
## [11] | boysen<berry>
## [19] | cloud<berry>
## [21] | cran<berry>
## [29] | elder<berry>
## [32] | goji <berry>
## [33] | goose<berry>
## [38] | huckle<berry>
## [50] | mul<berry>
## [70] | rasp<berry>
## [73] | salal <berry>
## [76] | straw<berry>
```

- **Literal characters:** Letters and numbers that match exactly.
- **Metacharacters:** Puntuation characters that have special meanings for pattern matching.
- The dot (`.`) metacharacter will match any string that contains a character that come before it:

```
str_view(c("a", "ab", "ae", "bd", "ea", "eab"), "a.")
```

```
## [2] | <ab>
## [3] | <ae>
## [6] | e<ab>
```

Example: Find all the the words that contain an `a` followed by three letters, followed by an `e` in `fruits`:

```
str_view(fruit, "a...e")
```

```
##  [1] | <apple>
##  [7] | bl<ackbe>rry
## [48] | mand<arine>
## [51] | nect<arine>
## [62] | pine<apple>
## [64] | pomegr<anate>
## [70] | r<aspbe>rry
## [73] | sal<al be>rry
```

Quantifiers: control how many times a pattern can match:
- `?`: makes a pattern optional (0 or once). - `+`: lets a pattern repeat (at least once). - `*`: lets a pattern be optional or repeat (any number of times, including 0).

Examples:

```
# matches an "a" optionally followed by a "b"
str_view(c("a", "ab", "abb"), "ab?")
```

```
## [1] | <a>
## [2] | <ab>
## [3] | <ab>b
```

```
# matches an "a" followed by at least one "b"
str_view(c("a", "ab", "abb"), "ab_")

# matches an "a" followed by any number of "b"s
str_view(c("a", "ab", "abb"), "ab*")
```

```
## [1] | <a>
## [2] | <ab>
## [3] | <abb>
```

- **Character classes:** defined by `[]` and match a set of characters. Starting with `^` matches everyting except the character class – the caret symbol acts like a "not" logic gate.

```
str_view(words, "[aeiou]x[aeiou]")
```

```
## [284] | <exa>ct
## [285] | <exa>mple
## [288] | <exe>rcise
## [289] | <exi>st
```

```
str_view(words, "[^aeiou]l[^aeiou]")
```

```
##  [45] | ap<ply>
## [253] | ea<rly>
## [328] | <fly>
## [578] | o<nly>
## [682] | rea<lly>
## [830] | sup<ply>
```

```
## [968] | wo<rld>
```

- Using the **alternation** symbol `|` to pick one or more alternative patterns:

```
str_view(fruit, "apple|melon|nut")
```

```
##  [1] | <apple>
## [13] | canary <melon>
## [20] | coco<nut>
## [52] | <nut>
## [62] | pine<apple>
## [72] | rock <melon>
## [80] | water<melon>
```

```
str_view(fruit, "aa|ee|ii|oo|uu")
```

```
##  [9] | bl<oo>d orange
## [33] | g<oo>seberry
## [47] | lych<ee>
## [66] | purple mangost<ee>n
```

Key functions

Detecting matches

- Detecting matches using `str_detect()`:

```
str_detect(c("a", "b", "c"), "[aeiou]")
```

```
## [1]  TRUE FALSE FALSE
```

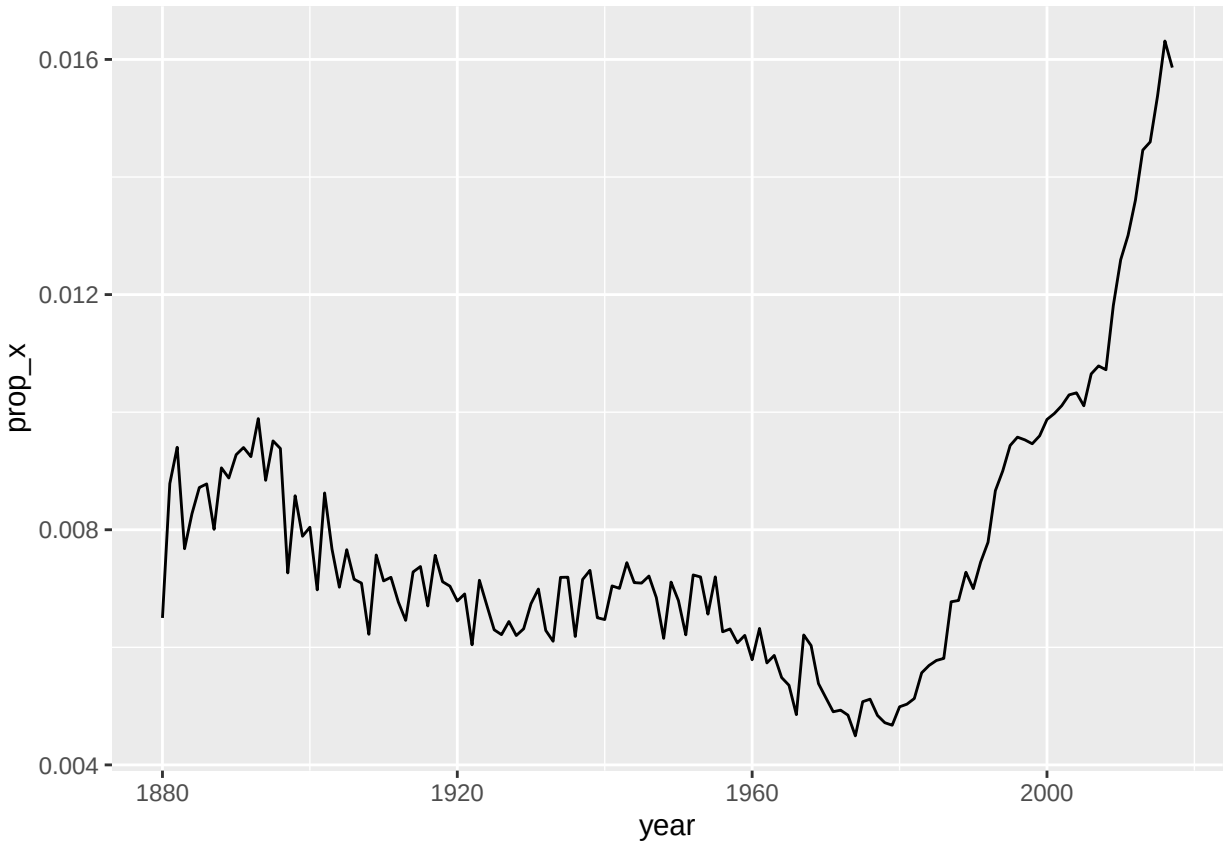
- Using `str_detect()` in dplyr:

```
babynames |>
  filter(str_detect(name, "x")) |>
  count(name, wt = n, sort = TRUE)
```

```
## # A tibble: 974 x 2
##    name            n
##    <chr>       <int>
##  1 Alexander  665492
##  2 Alexis     399551
##  3 Alex       278705
##  4 Alexandra  232223
##  5 Max        148787
##  6 Alexa      123032
##  7 Maxine     112261
##  8 Alexandria  97679
##  9 Maxwell     90486
## 10 Jaxon       71234
## # i 964 more rows
```

- Another example using `str_detect()` in dplyr (paired with `summarize()` this time):

```
babynames |>
  group_by(year) |>
  summarize(prop_x = mean(str_detect(name, "x"))) |>
  ggplot(aes(x = year, y = prop_x)) +
  geom_line()
```



Counting matches

- Using `str_count()` to determine how many matches:

```
x <- c("apple", "banana", "pear")

str_count(x, "p")
```

```
## [1] 2 0 1
```

- Regex matches never overlap – each match starts at the end of the previous match:

```
str_count("abababa", "aba")
```

```
## [1] 2
```

- Using `str_count()` in dplyr, paired with `mutate()`:

```
babynames |>
  count(name) |>
  mutate(
    vowels = str_count(name, "[aeiou]"),
    consonants = str_count(name, "[^aeiou]")
  )
```

```
## # A tibble: 97,310 x 4
##    name           n vowels consonants
##    <chr>      <int>  <int>      <int>
##  1 Aaban         10      2          3
##  2 Aabha          5      2          3
```

```
##  3 Aabid         2      2          3
##  4 Aabir         1      2          3
##  5 Aabriella     5      4          5
##  6 Aada          1      2          2
##  7 Aadam        26      2          3
##  8 Aadan        11      2          3
##  9 Aadarsh      17      2          5
## 10 Aaden        18      2          3
## # i 97,300 more rows
```

- The above result is not quite correct, since the default pattern matching is case-sensitive. Solution:

```
babynames |>
  count(name) |>
  mutate(
    vowels = str_count(name, "[aeiouAIEOU]"),
    consonants = str_count(name, "[^aeiouAEIOU]")
  )
```

```
## # A tibble: 97,310 x 4
##    name          n vowels consonants
##    <chr>     <int>  <int>      <int>
##  1 Aaban        10      3          2
##  2 Aabha         5      3          2
##  3 Aabid         2      3          2
##  4 Aabir         1      3          2
##  5 Aabriella     5      5          4
##  6 Aada          1      3          1
##  7 Aadam        26      3          2
##  8 Aadan        11      3          2
##  9 Aadarsh      17      3          4
## 10 Aaden        18      3          2
## # i 97,300 more rows
```

- Other applicable solutions: > Use `vowels = str_count(name, regex("[aeiou]", ignore_case = TRUE))` > Use ‘vowels = str_count(str_to_lower(name), “[aeiou]”)

```
babynames |>
  count(name) |>
  mutate(
    name = str_to_lower(name),
    vowels = str_count(name, "[aeiou]"),
    consonants = str_count(name, "[^aeiou]")
  )
```

```
## # A tibble: 97,310 x 4
##    name          n vowels consonants
##    <chr>     <int>  <int>      <int>
##  1 aaban        10      3          2
##  2 aabha         5      3          2
##  3 aabid         2      3          2
##  4 aabir         1      3          2
##  5 aabriella     5      5          4
##  6 aada          1      3          1
##  7 aadam        26      3          2
##  8 aadan        11      3          2
```

```
##  9 aadarsh      17      3          4
## 10 aaden        18      3          2
## # i 97,300 more rows
```

Replacing values

- Using `str_replace()` and/or `str_replace_all()`:

```
x <- c("apple", "pear", "banana")

str_replace_all(x, "[aeiou]", "-")
```

```
## [1] "-ppl-"  "p--r"   "b-n-n-"
```

- Using `str_remove()` and/or `str_remove_all()`:

```
x <- c("apple", "pear", "banana")

str_remove_all(x, "[aeiou]")
```

```
## [1] "ppl" "pr"  "bnn"
```

Extracting variables

- Using `separate_wider_` family of functions:

```
df <- tribble(
  ~str,
  "<Sheryl>-F_34",
  "<Kisha>-F_45",
  "<Brandon>-N_33",
  "<Sharon>-F_38",
  "<Penny>-F_58",
  "<Justin>-M_41",
  "<Patricia>-F_84",
)

df |>
  separate_wider_regex(
    str,
    patterns = c(
      "<",
      name = "[A-Za-z]+",
      ">-",
      gender = ".",
      "_",
      age = "[0-9]+"
    )
  )
```

```
## # A tibble: 7 x 3
##   name     gender age
##   <chr>    <chr>  <chr>
## 1 Sheryl   F      34
## 2 Kisha    F      45
## 3 Brandon  N      33
## 4 Sharon   F      38
```

```
## 5 Penny    F      58
## 6 Justin   M      41
## 7 Patricia F      84
```

Exercises

1. What baby name has the most vowels? What name has the highest proportion of vowels? (Hint: what is the denominator?)

Ans.

baby name with the most vowels:

```
babynames |>
  mutate(
    name = str_to_lower(name),
    vowels = str_count(name, "[aeiou]")
  ) |>
  filter(
    vowels == max(vowels)) |>
  distinct(name, .keep = vowels) |>
  rename(vowels = .keep)
```

```
## # A tibble: 2 x 2
##   name            vowels
##   <chr>            <int>
## 1 mariaguadalupe       8
## 2 mariadelrosario      8
```

Baby name with the highest proportion of vowels:

```
babynames |>
  mutate(
    name = str_to_lower(name),
    vowels = str_count(name, "[aeiou]"),
    length = str_length(name),
    v_prop = vowels / length
  ) |>
  arrange(desc(v_prop)) |>
  distinct(name, .keep_all = TRUE) |>
  select(name, vowels, v_prop)
```

```
## # A tibble: 97,310 x 3
##    name   vowels v_prop
##    <chr>   <int>  <dbl>
##  1 eua         3  1
##  2 ea          2  1
##  3 ai          2  1
##  4 ia          2  1
##  5 ii          2  1
##  6 aoi         3  1
##  7 io          2  1
##  8 aia         3  1
##  9 alaiia      5  0.833
## 10 amaiia      5  0.833
## # i 97,300 more rows
```

2. Replace all forward slashes in "a/b/c/d/e" with backslashes. What happens if you attempt to undo the transformation by replacing all backslashes with forward slashes? (We'll discuss the problem soon.)

```
x2 <- "a/b/c/d/e"

str_replace(x2, "/", "\\")
```

```
## [1] "ab/c/d/e"
```

This won't work as of now because slash is a special character. We need to use escaping keys.

3. Implement a simple version of `str_to_lower()` using `str_replace_all()`.

```
x3 <- "thE qUIck And brOwn fOx"

x3 <- str_replace_all(x3, "[A]", "a")
x3 <- str_replace_all(x3, "[E]", "e")
x3 <- str_replace_all(x3, "[I]", "i")
x3 <- str_replace_all(x3, "[O]", "o")
x3 <- str_replace_all(x3, "[U]", "u")

str_view(x3)
```

```
## [1] | the quick and brown fox
```

4. Create a regular expression that will match telephone numbers as commonly written in your country.

```
text = "lkjsdfljfds098234kljsd+63-927-5589flkjsdf09usdflkjsdflkjsdf"

pattern = "[+]63-[0123456789][0123456789][0123456789]-[0123456789][0123456789][0123456789][0123456789]"
str_view(text, pattern)
```

```
## [1] | lkjsdfljfds098234kljsd<+63-927-5589>flkjsdf09usdflkjsdflkjsdf
```

Pattern details

Escaping

- Escaping using backslash \:

```
# to match a "." we need to use "\\."
dot <- "\\."
str_view(dot)
```

```
## [1] | \.
```

```
str_view(c("abc", "a.c"), "a\\.c")
```

```
## [2] | <a.c>
```

- To match a literal backslash (\), we need to use four backslashes (\\\\):

```
x <- "a\\b"
str_view(x)
```

```
## [1] | a\b
```

```
str_view(x, "\\\\")
```

```
## [1] | a<\>b
```

- Using raw strings in this case lets us avoid one layer of escaping:

```
str_view(x, r"{\\}")
```

```
## [1] | a<\>b
```

- To match a literal character that is also a metacharacter (`.`, `$`, etc.), it is more practical to use a character class (`[.]`, `[&]`, etc.)

```
str_view(c("abc", "a.c", "a*c", "a c"), "a[.]c")
```

```
## [2] | <a.c>
```

```
str_view(c("abc", "a.c", "a*c", "a c"), ".[*]c")
```

```
## [3] | <a*c>
```

Anchors

- Used if you want to match at the start or end of the string. Default is to match any part of the string.

```
# match at the start of the string
str_view(fruit, "^a")
```

```
## [1] | <a>pple
## [2] | <a>pricot
## [3] | <a>vocado
```

```
# match at the end
str_view(fruit, "a$")
```

```
##  [4] | banan<a>
## [15] | cherimoy<a>
## [30] | feijo<a>
## [36] | guav<a>
## [56] | papay<a>
## [74] | satsum<a>
```

- To force a regex to match only the full string, anchor it with both `^` and `$`:

```
str_view(fruit, "apple")
```

```
##  [1] | <apple>
## [62] | pine<apple>
```

```
str_view(fruit, "^apple$")
```

```
## [1] | <apple>
```

- To match the boundaries between words, `\b` is used. This works for the start or end of a word:

```
x <- c("summary(x)", "summarize(df", "rowsum(x)", "sum(x)")
str_view(x, "sum")
```

```
## [1] | <sum>mary(x)
## [2] | <sum>marize(df
## [3] | row<sum>(x)
## [4] | <sum>(x)
```

```
str_view(x, "\\bsum\\b")
```

```
## [4] | <sum>(x)
```

- When used alone, anchors will produce a zero-width match:

```
str_view("abc", c("S", "^", "\\b"))
```

```
## [2] | <>abc
## [3] | <>abc<>
```

- Replacing a standalone anchor:

```
str_replace_all("abc", c("$", "^", "\\b"), "--")
```

```
## [1] "abc--"   "--abc"   "--abc--"
```

Character class shorthand

- `-` defines a range. Example: `[a-z]` and `[0-9]`.
- `\` escapes special characters. Example: `[\^\-\]]` matches `^`, `-`, or `]`.

Example with code:

```
x <- "abcd ABCD 12345 -!@#%."

str_view(x, "[abc]+")
```

```
## [1] | <abc>d ABCD 12345 -!@#%.
```

```
str_view(x, "[a-z0-9]+")
```

```
## [1] | <abcd> ABCD <12345> -!@#%.
```

```
str_view(x, "[^a-z0-9]+")
```

```
## [1] | abcd< ABCD >12345< -!@#%.>
```

```
str_view("a-b-c", "[a-c]")
```

```
## [1] | <a>-<b>-<c>
```

```
str_view("a-b-c", "[a\\-c]")
```

```
## [1] | <a><->b<-><c>
```

Other shorthands: - `\d` matches any digit.
- `\D` matches anything that isn't a digit.
- `\s` matches any whitespace.
- `\S` matches anything that isn't a whitespace.
- `\w` matches any "word" character.
- `\W` matches any "non-word" character.

Examples:

```
x <- "abcd ABCD 12345 -!@#$."

str_view(x, "\\d+")
```

```
## [1] | abcd ABCD <12345> -!@#$.
```

```
str_view(x, "\\D+")
```

```
## [1] | <abcd ABCD >12345< -!@#$.>
```

```
str_view(x, "\\s+")
```

```
## [1] | abcd< >ABCD< >12345< >-!@#$.
```

```
str_view(x, "\\S+")
```

```
## [1] | <abcd> <ABCD> <12345> <-!@#$.>
```

```
str_view(x, "\\w+")
```

```
## [1] | <abcd> <ABCD> <12345> -!@#$.
```

```
str_view(x, "\\W+")
```

```
## [1] | abcd< >ABCD< >12345< -!@#$.>
```

Quantifiers

- Quantifiers control how many times a pattern matches.
- `{n}` matches exactly n times.
- `{n,}` matches at least n times.
- `{n, m}` matches between n and m times.

On the order of operation: - Quantifiers have high precedence.
- Alternation has low precedence.

Grouping and capturing

- Capturing groups allow the user to use sub-components of the match.
- **back reference:**: `\1` refers to the match contained in the first parenthesis, `\2` in the second parenthesis, and so on.
- Example using back reference to find all fruits that have a repeated pair of letters:

```
str_view(fruit, "(..)\\1")
```

```
##  [4] | b<anan>a
## [20] | <coco>nut
## [22] | <cucu>mber
## [41] | <juju>be
## [56] | <papa>ya
## [73] | s<alal> berry
```

- Example using back reference to find all words that start and end with the same pair of letters:

```
str_view(words, "^(..).*\\1$")
```

```
## [152] | <church>
## [217] | <decide>
## [617] | <photograph>
## [699] | <require>
## [739] | <sense>
```

- Using back references in `str_replace()`:

```
sentences |>
  str_replace("(\\w+) (\\w+) (\\w+)", "\\1 \\3 \\2") |>
  str_view()
```

```
##  [1] | The canoe birch slid on the smooth planks.
##  [2] | Glue sheet the to the dark blue background.
##  [3] | It's to easy tell the depth of a well.
##  [4] | These a days chicken leg is a rare dish.
##  [5] | Rice often is served in round bowls.
##  [6] | The of juice lemons makes fine punch.
##  [7] | The was box thrown beside the parked truck.
##  [8] | The were hogs fed chopped corn and garbage.
##  [9] | Four of hours steady work faced us.
## [10] | A size large in stockings is hard to sell.
## [11] | The was boy there when the sun rose.
## [12] | A is rod used to catch pink salmon.
## [13] | The of source the huge river is the clear spring.
## [14] | Kick ball the straight and follow through.
## [15] | Help woman the get back to her feet.
## [16] | A of pot tea helps to pass the evening.
## [17] | Smoky lack fires flame and heat.
## [18] | The cushion soft broke the man's fall.
## [19] | The breeze salt came across from the sea.
## [20] | The at girl the booth sold fifty bonds.
## ... and 700 more
```

- Extracting matches for each group:

```
sentences |>
  str_match("the (\\w+) (\\w+)") |>
  head()
```

```
##      [,1]                [,2]     [,3]
## [1,] "the smooth planks" "smooth" "planks"
## [2,] "the sheet to"      "sheet"  "to"
## [3,] "the depth of"      "depth"  "of"
## [4,] NA                  NA       NA
## [5,] NA                  NA       NA
## [6,] NA                  NA       NA
```

- Converting the above result to a tibble:

```
sentences |>
  str_match("the (\\w+) (\\w+)") |>
  as_tibble(.name_repair = "minimal") |>
  set_names("match", "word1", "word2")
```

```
## # A tibble: 720 x 3
##    match             word1  word2
##    <chr>             <chr>  <chr>
##  1 the smooth planks smooth planks
##  2 the sheet to      sheet  to
##  3 the depth of      depth  of
##  4 <NA>              <NA>   <NA>
##  5 <NA>              <NA>   <NA>
##  6 <NA>              <NA>   <NA>
##  7 the parked truck  parked truck
##  8 <NA>              <NA>   <NA>
##  9 <NA>              <NA>   <NA>
## 10 <NA>              <NA>   <NA>
## # i 710 more rows
```

- Using `(?:)` to have parentheses without creating matching groups (aka non-capturing group):

```
x <- c("a gray cat", "a grey dog")

# with capturing group
str_match(x, "gr(e|a)y")
```

```
##      [,1]   [,2]
## [1,] "gray" "a"
## [2,] "grey" "e"
```

```
# with non-capturing group
str_match(x, "gr(?:e|a)y")
```

```
##      [,1]
## [1,] "gray"
## [2,] "grey"
```

Exercises

...

Pattern control

- Used when extra control over the details of the match is needed.

Regex flags

- `ignore_case = TRUE` flag is probably the most useful because it allows characters to match both uppercase or lowercase forms.

```
bananas <- c("banana", "Banana", "BANANA")

# no regex flag
str_view(bananas, "banana")
```

```
## [1] | <banana>
```

```
# with `ignore_case = TRUE` flag
str_view(bananas, regex("banana", ignore_case = TRUE))
```

```
## [1] | <banana>
## [2] | <Banana>
## [3] | <BANANA>
```

- Flags for working with multiline strings:
- `dotall = TRUE` lets `.` match everything including `\n`.

```
x <- "Line 1\nLine 2\nLine 3"

# no flag
str_view(x, ".Line")

# with `dotall = TRUE` flag
str_view(x, regex(".Line", dotall = TRUE))
```

```
## [1] | Line 1<
##     | Line> 2<
##     | Line> 3
```

- `multiline = TRUE` makes ^ and & match the start and end of each line rather than those of the complete string:

```
x <- "Line 1\nLine 2\nLine 3"

# no flag
str_view(x, "^Line")
```

```
## [1] | <Line> 1
##     | Line 2
##     | Line 3
```

```
# with `multiline = TRUE` flag
str_view(x, regex("^Line", multiline = TRUE))
```

```
## [1] | <Line> 1
##     | <Line> 2
##     | <Line> 3
```

- `comments = TRUE` allows the use of comments and whitespace to make complex regex more understandable:

```
phone <- regex(
  r"(
    \(?     # optional opening parens
    (\d{3}) # area code
    [)\-]?  # optional closing parens or dash
    \ ?     # optional space
    (\d{3}) # another three numbers
    [\ -]?  # optional space or dash
    (\d{4}) # four more numbers
  )",
  comments = TRUE
)

str_extract(c("514-791-8141", "(123) 456 7890", "123456"), phone)
```

```
## [1] "514-791-8141"   "(123) 456 7890" NA
```

Fixed matches

- `fixed()` is used to opt-out of the regex rules:

```
str_view(c("", "a", "."), fixed("."))
```

```
## [3] | <.>
```

- `fixed()` can also be used to ignore cases:

```
str_view("x X", "X")
```

```
## [1] | x <X>
```

```
str_view("x X", fixed("X", ignore_case = TRUE))
```

```
## [1] | <x> <X>
```

- when working with non-English text, `coll()` works better than `fixed()`:

```
str_view("i İ ı I", fixed("İ", ignore_case = TRUE))
```

```
## [1] | i <İ> ı I
```

```
str_view("i İ ı I", coll("İ", ignore_case = TRUE, locale = "tr"))
```

```
## [1] | <i> <İ> ı I
```

Practice

Check your work

Exercise: Finding all sentences that start with "The":

```
str_view(sentences, "^The")
```

```
##  [1] | <The> birch canoe slid on the smooth planks.
##  [4] | <The>se days a chicken leg is a rare dish.
##  [6] | <The> juice of lemons makes fine punch.
##  [7] | <The> box was thrown beside the parked truck.
##  [8] | <The> hogs were fed chopped corn and garbage.
## [11] | <The> boy was there when the sun rose.
## [13] | <The> source of the huge river is the clear spring.
## [18] | <The> soft cushion broke the man's fall.
## [19] | <The> salt breeze came across from the sea.
## [20] | <The> girl at the booth sold fifty bonds.
## [21] | <The> small pup gnawed a hole in the sock.
## [22] | <The> fish twisted and turned on the bent hook.
## [24] | <The> swan dive was far short of perfect.
## [25] | <The> beauty of the view stunned the young boy.
## [28] | <The> colt reared and threw the tall rider.
## [36] | <The> wrist was badly strained and hung limp.
## [37] | <The> stray cat gave birth to kittens.
## [38] | <The> young girl gave no clear response.
## [39] | <The> meal was cooked before the bell rang.
## [42] | <The> ship was torn apart on the sharp reef.
## ... and 257 more
```

- We can see that this is not good enough. Words like `These` and `They` are also matches.
- Adding a word boundary:

```
str_view(sentences, "^The\\b")
```

```
##  [1] | <The> birch canoe slid on the smooth planks.
##  [6] | <The> juice of lemons makes fine punch.
##  [7] | <The> box was thrown beside the parked truck.
##  [8] | <The> hogs were fed chopped corn and garbage.
## [11] | <The> boy was there when the sun rose.
## [13] | <The> source of the huge river is the clear spring.
## [18] | <The> soft cushion broke the man's fall.
## [19] | <The> salt breeze came across from the sea.
## [20] | <The> girl at the booth sold fifty bonds.
## [21] | <The> small pup gnawed a hole in the sock.
## [22] | <The> fish twisted and turned on the bent hook.
## [24] | <The> swan dive was far short of perfect.
## [25] | <The> beauty of the view stunned the young boy.
## [28] | <The> colt reared and threw the tall rider.
## [36] | <The> wrist was badly strained and hung limp.
## [37] | <The> stray cat gave birth to kittens.
```

```
## [38] | <The> young girl gave no clear response.
## [39] | <The> meal was cooked before the bell rang.
## [42] | <The> ship was torn apart on the sharp reef.
## [44] | <The> wide road shimmered in the hot sun.
## ... and 236 more
```

Looks OK!

- Exercise: Finding all sentences that begin with a pronoun:

```
str_view(sentences, "^She|he|It|They\\b")
```

```
##  [1] | T<he> birch canoe slid on t<he> smooth planks.
##  [2] | Glue t<he> s<he>et to t<he> dark blue background.
##  [3] | <It>'s easy to tell t<he> depth of a well.
##  [4] | T<he>se days a chicken leg is a rare dish.
##  [6] | T<he> juice of lemons makes fine punch.
##  [7] | T<he> box was thrown beside t<he> parked truck.
##  [8] | T<he> hogs were fed chopped corn and garbage.
## [11] | T<he> boy was t<he>re w<he>n t<he> sun rose.
## [13] | T<he> source of t<he> huge river is t<he> clear spring.
## [14] | Kick t<he> ball straight and follow through.
## [15] | Help t<he> woman get back to <he>r feet.
## [16] | A pot of tea <he>lps to pass t<he> evening.
## [17] | Smoky fires lack flame and <he>at.
## [18] | T<he> soft cushion broke t<he> man's fall.
## [19] | T<he> salt breeze came across from t<he> sea.
## [20] | T<he> girl at t<he> booth sold fifty bonds.
## [21] | T<he> small pup gnawed a hole in t<he> sock.
## [22] | T<he> fish twisted and turned on t<he> bent hook.
## [23] | Press t<he> pants and sew a button on t<he> vest.
## [24] | T<he> swan dive was far short of perfect.
## ... and 542 more
```

This is not good enough.

- It is a good idea here to create a positive and negative matches and use them to test if your pattern works as expected:

```
pos <- c("He is a boy.", "She had a good time")
neg <- c("Shells come from the sea", "Hadley said 'It's a great day'")

pattern <- "^(She|He|It|They)\\b"
str_detect(pos, pattern)
```

```
## [1] TRUE TRUE
```

```
str_detect(neg, pattern)
```

```
## [1] FALSE FALSE
```

Boolean operations

Example: Find words that only contain consonants

```
str_view(words, "^[^aeiou]+$")
```

```
## [123] | <by>
## [249] | <dry>
```

```
## [328] | <fly>
## [538] | <mrs>
## [895] | <try>
## [952] | <why>
```

- This works but there is an easier way – by flipping the problem around using the "not" operator.

- Instead of looking for consonants, we can look for words that don't contain any vowels:

```
str_view(words[!str_detect(words, "[aeiou]")])
```

```
## [1] | by
## [2] | dry
## [3] | fly
## [4] | mrs
## [5] | try
## [6] | why
```

Example: Find all words that contain "a" and "b". - Plan of attack: look for all words that contain an "a" followed by a "b" or a "b" followed by an "a":

```
str_view(words, "a.*b|b.*a")
```

```
##   [2] | <ab>le
##   [3] | <ab>out
##   [4] | <ab>solute
##  [62] | <availab>le
##  [66] | <ba>by
##  [67] | <ba>ck
##  [68] | <ba>d
##  [69] | <ba>g
##  [70] | <bala>nce
##  [71] | <ba>ll
##  [72] | <ba>nk
##  [73] | <ba>r
##  [74] | <ba>se
##  [75] | <ba>sis
##  [77] | <bea>r
##  [78] | <bea>t
##  [79] | <bea>uty
##  [80] | <beca>use
##  [95] | <bla>ck
## [100] | <boa>rd
## ... and 10 more
```

- This plan works but is rather difficult to come up with. There is an easier way:

```
words[str_detect(words, "a") & str_detect(words, "b")]
```

```
##  [1] "able"      "about"     "absolute"  "available" "baby"      "back"
##  [7] "bad"       "bag"       "balance"   "ball"      "bank"      "bar"
## [13] "base"      "basis"     "bear"      "beat"      "beauty"    "because"
## [19] "black"     "board"     "boat"      "break"     "brilliant" "britain"
## [25] "debate"    "husband"   "labour"    "maybe"     "probable"  "table"
```

Example: Try to see if there was a word that contains all vowels.
- Using the first plan of attack is not practical, since this requires 120 different patterns (5!).
- a better way is to use the second approach:

```
words[
  str_detect(words, "a") &
  str_detect(words, "e") &
  str_detect(words, "i") &
  str_detect(words, "o") &
  str_detect(words, "u")
]
```

```
## character(0)
```

- If you get stuck trying to create a single regex pattern that solves your problem, take a step back and think if you could break the problem down into smaller pieces.

Creating a pattern with code

Problem: Find all sentences that mention a color.

```
str_view(sentences, "\\b(red|green|blue)\\b")
```

```
##   [2] | Glue the sheet to the dark <blue> background.
##  [26] | Two <blue> fish swam in the tank.
##  [92] | A wisp of cloud hung in the <blue> air.
## [148] | The spot on the blotter was made by <green> ink.
## [160] | The sofa cushion is <red> and of light weight.
## [174] | The sky that morning was clear and bright <blue>.
## [204] | A <blue> crane is a tall wading bird.
## [217] | It is hard to erase <blue> or <red> ink.
## [224] | The lamp shone with a steady <green> flame.
## [247] | The box is held by a bright <red> snapper.
## [256] | The houses are built of <red> clay bricks.
## [274] | The <red> tape bound the smuggled food.
## [288] | Hedge apples may stain your hands <green>.
## [302] | The plant grew large and <green> in the window.
## [330] | Bathe and relax in the cool <green> grass.
## [368] | The lake sparkled in the <red> hot sun.
## [372] | Mark the spot with a sign painted <red>.
## [452] | The couch cover and hall drapes were <blue>.
## [491] | A man in a <blue> sweater sat at the desk.
## [551] | The small <red> neon lamp went out.
## ... and 6 more
```

What if we have more colors? Things would get cumbersome very quickly.

- Better solution: store the colors in a vector and use `str_c` and `str_flatten()` to generate the necessary pattern.

```
rgb <- c("red", "green", "blue")

str_c("\\b(", str_flatten(rgb, "|"), ")\\b")
```

```
## [1] "\\b(red|green|blue)\\b"
```

- It works, but where can we get a list of colors?

- There is a color list in R, but some items are numbered.

```
str_view(colors())
```

```
##  [1] | white
##  [2] | aliceblue
##  [3] | antiquewhite
##  [4] | antiquewhite1
##  [5] | antiquewhite2
##  [6] | antiquewhite3
##  [7] | antiquewhite4
##  [8] | aquamarine
##  [9] | aquamarine1
## [10] | aquamarine2
## [11] | aquamarine3
## [12] | aquamarine4
## [13] | azure
## [14] | azure1
## [15] | azure2
## [16] | azure3
## [17] | azure4
## [18] | beige
## [19] | bisque
## [20] | bisque1
## ... and 637 more
```

- Eliminating the numbered colors:

```
cols <- colors()
cols <- cols[!str_detect(cols, "\\d")]
str_view(cols)
```

```
##  [1] | white
##  [2] | aliceblue
##  [3] | antiquewhite
##  [4] | aquamarine
##  [5] | azure
##  [6] | beige
##  [7] | bisque
##  [8] | black
##  [9] | blanchedalmond
## [10] | blue
## [11] | blueviolet
## [12] | brown
## [13] | burlywood
## [14] | cadetblue
## [15] | chartreuse
## [16] | chocolate
## [17] | coral
## [18] | cornflowerblue
## [19] | cornsilk
## [20] | cyan
## ... and 123 more
```

Finally we can use this to generate the pattern:

```
pattern <- str_c("\\b(", str_flatten(cols, "|"), ")\\b")
str_view(sentences, pattern)
```

```
##   [2] | Glue the sheet to the dark <blue> background.
```

```
##  [12] | A rod is used to catch <pink> <salmon>.
##  [26] | Two <blue> fish swam in the tank.
##  [66] | Cars and busses stalled in <snow> drifts.
##  [92] | A wisp of cloud hung in the <blue> air.
## [112] | Leaves turn <brown> and <yellow> in the fall.
## [148] | The spot on the blotter was made by <green> ink.
## [149] | Mud was spattered on the front of his <white> shirt.
## [160] | The sofa cushion is <red> and of light weight.
## [167] | The office paint was a dull, sad <tan>.
## [174] | The sky that morning was clear and bright <blue>.
## [201] | The <brown> house was on fire to the attic.
## [204] | A <blue> crane is a tall wading bird.
## [217] | It is hard to erase <blue> or <red> ink.
## [221] | A pencil with <black> lead writes best.
## [224] | The lamp shone with a steady <green> flame.
## [230] | Slash the <gold> cloth into fine ribbons.
## [234] | They floated on the raft to sun their <white> backs.
## [246] | The birch looked stark <white> and lonesome.
## [247] | The box is held by a bright <red> snapper.
## ... and 43 more
```

Exercises

...

Regex in other places

tidyverse

Some useful places where regex is useful - `matches(pattern)` will select all available variables whose name matches the supplied pattern. Abvailable in `select()`, `rename_with()`, and `across()`.
- `names_pattern` argument in `pivot_longer()`. Useful for extracting data out of variable names with complex structure.
- `delim` argument in `separate_longer_delim()` and family of functions.

Base R

Some useful applications of regex in base R: - `apropos(pattern)` searches all objects available in the global environment. Useful if you can't quite remember the name of a function.
- `list.files(path, pattern)` lists all files in `path` that math a regex pattern.